UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

SIS_ROPA 2.0

Sistema POS y tienda en línea con arquitectura de microservicios

Proyectos finales 1 y 2 · Desarrollo de Aplicaciones Web

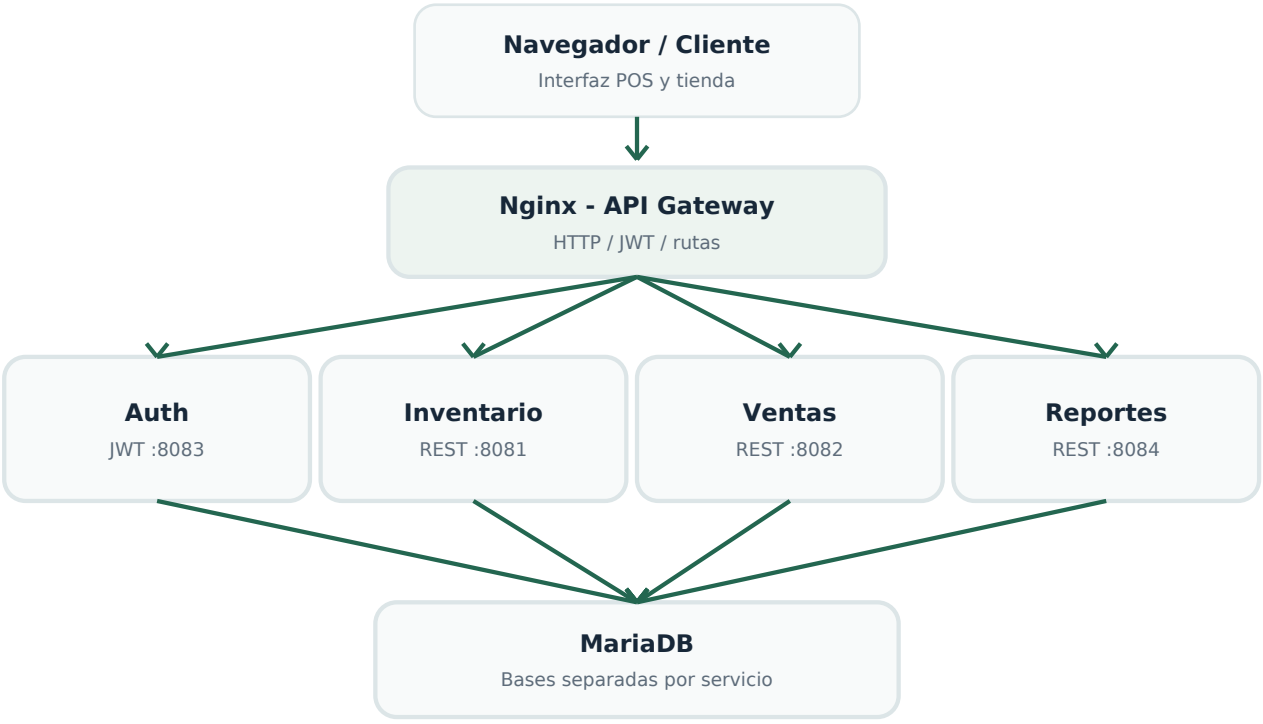
Tecnologías: Laravel, PHP 8.4, MariaDB, Nginx, Docker, API REST, JWT y servicios externos.

INFORMACIÓN	DETALLE
Institución	Universidad Mariano Gálvez de Guatemala
Carrera	Ingeniería en Sistemas de Información y Ciencias de la Computación
Fecha de entrega	31 de octubre de 2026
Estudiante	_____
Repositorio y URL pública	Completar al publicar la rama de entrega

Documento técnico basado en evidencia local proporcionada por el estudiante. No acredita por sí solo que el despliegue distribuido esté publicado en internet.

01 Arquitectura del sistema

El proyecto implementa cuatro procesos PHP independientes, un gateway Nginx y bases de datos separadas por servicio. La instalación local cuenta, además, con MariaDB principal y réplica, y dos instancias de inventario para balanceo de carga.



Dos caminos operativos distintos

Aplicación comercial Laravel: las ventas, compras y pedidos utilizan la base pos_ropa y envían proyecciones de consulta a los servicios REST. API distribuida de demostración: registra sus propias ventas y solicita descuentos de inventario mediante REST. La prueba de ambos caminos no significa que sean una transacción comercial única.

02 Funcionalidades e integraciones

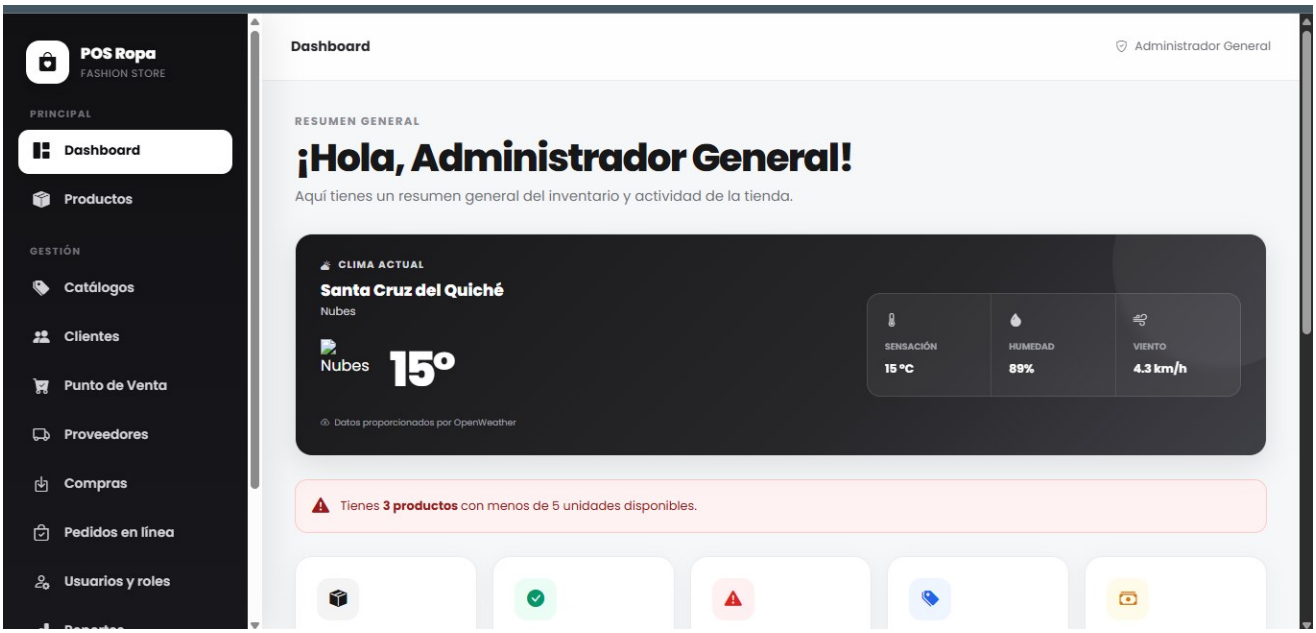
MÓDULO	CARACTERÍSTICAS
Punto de venta	Carrito, variantes talla/color, descuentos, facturas, IVA y QR
Tienda web	Catálogo, registro de clientes, carrito, pedidos y checkout Stripe sandbox
Inventario	21 productos de prueba; categorías, marcas, tallas, colores y alertas
Administración	Usuarios/roles, proveedores, compras, pedidos y reportes Chart.js
API y seguridad	JWT para microservicios; Laravel Sanctum para API de aplicación

APIs de terceros

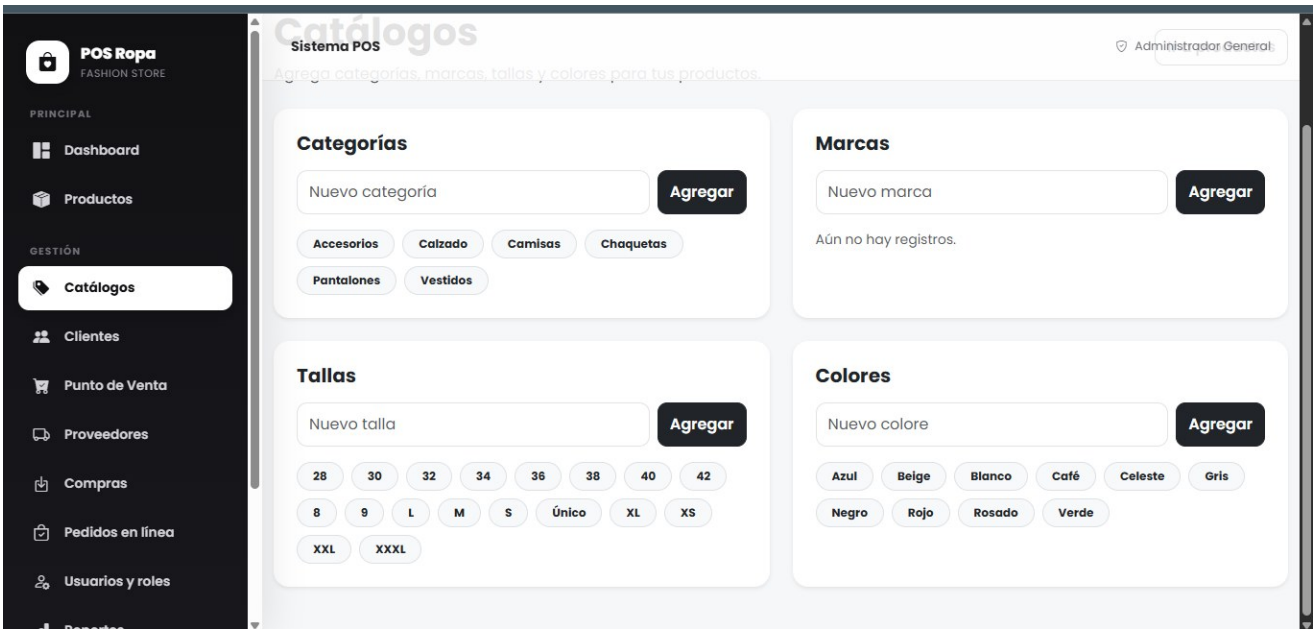
API	ESTADO VERIFICADO
Stripe Sandbox	Opción de pago con tarjeta comprobada localmente; verificar pago web y webhook públicos
OpenWeather	Panel local muestra temperatura, humedad y viento en Santa Cruz del Quiché
QR Server API	Se generan URLs para el código QR de la factura
Cloudinary	Integración de código preparada; falta evidencia de subida real a la nube

03 Evidencias de interfaz

Dashboard con datos de OpenWeather



Módulo de catálogos de productos



Capturas de SIS_ROPA ejecutándose localmente en Docker, puerto 8090. Las imágenes no demuestran disponibilidad pública del servidor.

04 Pruebas técnicas y resultados

PRUEBA	RESULTADO
Contenedores de aplicación y servicios	Iniciados en Docker Desktop
JWT / protección de rutas	Login y verificación válidos; HTTP 401 sin token
MariaDB principal y réplica	21 productos; 225 unidades; datos coincidentes en auditoría local
Sincronización Laravel a REST	1 venta POS y 2 pedidos web registrados en evidencia del 28/10
Balanceo Nginx	8 solicitudes distribuidas entre inventory e inventory2
Tolerancia a fallos	inventory2 respondió al detener inventory; después se restauró
Nuevos endpoints de factura y reportes	Implementados y comprobados por sintaxis; prueba funcional pendiente en Docker
Publicación de microservicios en Railway	Pendiente de verificar URL, variables, volumen y servicios públicos

Alcance y precauciones

El funcionamiento local demuestra la arquitectura, pero no reemplaza la evidencia de despliegue público. Debe probarse el flujo real de Stripe con tarjeta sandbox, Cloudinary y, si se configura, webhook firmado. Las credenciales y claves privadas no deben formar parte de esta documentación.

05 API REST y entrega

MÉTODO Y RUTA	FUNCIÓN
POST /api/auth/login	Emitir JWT para el sistema distribuido
POST /api/auth/verify	Comprobar autenticación JWT
GET /api/inventario/pos/productos	Inventario comercial proyectado
GET /api/inventario/pos/resumen	Resumen de existencias
POST /api/ventas	Venta independiente de demostración REST
GET /api/ventas/{id}/factura	Comprobante REST / QR
GET /api/reportes/pos/ventas	Ventas POS proyectadas filtradas por fecha
GET /api/reportes/top-productos	Top de artículos vendidos
GET /api/reportes/pos/resumen	Reporte consolidado por REST

Material que debe acompañar la entrega

• Enlace del repositorio GitHub y URL pública accesible.

• Cuenta de administrador, cajero y cliente de prueba comunicadas privadamente al catedrático.

• Capturas reales de pago, carrito, catálogo, dashboard, reportes y servicios Railway.

• Resultados TXT de auditoría, balanceo y pruebas de endpoints en docs/evidencias.

• Archivo README/guía de despliegue y demostración práctica de 10 minutos.

Responsabilidad de publicación

Los archivos de despliegue incluidos son una propuesta revisada estáticamente. La configuración final requiere crear servicios, contraseñas y volúmenes en Railway, y comprobar cada servicio desde una URL HTTPS real.